

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I RAJESWARI P-192311072 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
RAJESWARI P

Annexure A

Reverse Engineering

1. Reverse Engineering a Recommendation System

A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

2. Reverse Engineering a Smart Traffic Management System

A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

3. Reverse Engineering a Fraud Detection Pipeline

A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

4. Reverse Engineering a Personalized Learning System

An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

ANSWER

1. Reverse Engineering a Recommendation System

A large-scale e-commerce recommendation system collects data from user searches, clicks, purchases, ratings, and browsing history. The collected data is stored in distributed systems and processed using PySpark RDDs. RDD operations such as `map()`, `filter()`, `flatMap()`, `groupByKey()`, and `reduceByKey()` transform raw user-item data into useful features. Collaborative filtering identifies similar users and products, while content-based filtering compares product characteristics with user preferences. Intelligent agents continuously monitor user behavior and feedback and update recommendation scores dynamically. The final recommendations are ranked and delivered to users in real time. Distributed processing allows millions of interactions to be processed efficiently.

2. Reverse Engineering a Smart Traffic Management System

A smart traffic management system collects real-time information from cameras, road sensors, GPS devices, and connected vehicles. The data is continuously processed through a streaming pipeline using PySpark. Incoming data can be represented as RDDs and processed using operations such as `map()`, `filter()`, `reduceByKey()`, and `join()`. The system calculates vehicle density, traffic speed, congestion levels, and waiting time. Reactive traffic agents monitor these values and communicate with signal-control agents. When congestion increases, the agents dynamically modify traffic signal timings. Decisions must be made within a short time to respond to changing traffic conditions. This distributed architecture enables real-time traffic monitoring and congestion reduction.

3. Reverse Engineering a Fraud Detection Pipeline

A banking fraud detection system receives transaction data such as amount, location, time, account information, and transaction frequency. PySpark RDDs distribute the transactions across multiple processing nodes. RDD transformations such as `map()` and `filter()` extract important transaction features, while `groupByKey()` and `reduceByKey()` help identify unusual transaction patterns. Anomaly detection algorithms compare transactions with normal customer behavior and calculate a fraud score. Intelligent agents continuously monitor transactions and coordinate the detection process. If a transaction exceeds a predefined risk level, an alert agent generates a warning for further investigation. Feedback from confirmed fraudulent and legitimate transactions can be used to improve the detection model.

4. Reverse Engineering a Personalized Learning System

A personalized learning platform collects student data such as quiz scores, assignment marks, learning time, completed lessons, and engagement levels. PySpark RDDs distribute and process this information efficiently. Operations such as `map()`, `filter()`, `groupByKey()`, and `reduceByKey()` are used to calculate student performance and learning patterns. A learning agent analyzes the results and identifies each student's strengths and weaknesses. Based on performance, the agent adjusts content difficulty and recommends suitable learning materials. A feedback agent monitors student responses and updates future recommendations. Continuous analysis allows the system to provide adaptive content, personalized feedback, and improved learning paths for large numbers of students.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud-based AI monitoring system collects large volumes of server logs, CPU usage, memory usage, network traffic, application events, and system errors. PySpark processes these logs using distributed RDDs. Operations such as `map()`, `filter()`, `reduceByKey()`, and `join()` extract important metrics and identify abnormal patterns. RDDs provide fault tolerance because lost partitions can be recomputed from their original transformations. Anomaly-detection agents monitor system behavior and identify unusual resource usage or failures. Scaling agents analyze workload levels and increase or decrease cloud resources when required. Response agents coordinate actions such as generating alerts, restarting services, or allocating additional resources. This architecture provides scalable, fault-tolerant, and real-time cloud monitoring.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context

Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation